CU HELPDESK — YOUR ACADEMIC SUPPORT COMMUNITY

Logical Thinking & Problem Solving

Complete C Programming Master Notes — Lectures 1 to 12

SUBJECT CODE
26CSH-101 / 26CSH-1071

SEMESTER / BRANCH
1st Semester B.Tech CS & Engineering

TARGET
Mid-Sem & End-Sem Exam Ready

NOTES TYPE
Full Package: Theory + Code + Practice + Revision

Scanned with CamScanner

Table of Contents

1. Introduction to C Programming & Compilation
2. Flowcharts & Visual Problem Solving
3. Algorithms & Performance Analysis
4. Memory Layout of a C Program
5. Input / Output Functions in C
6. Data Types, Variables & Keywords
7. Operators in C (Arithmetic, Relational, Logical)
8. Bitwise Operators & Type Conversion
9. C Expressions & Operator Precedence
10. Decision Control Statements (if-else)
11. Switch Case Statement
12. Conditional Operator & Goto Statement

Scanned with CamScanner

CU Helpdesk — Your Academic Support Community

Click Here to Join WhatsApp Group

01 Lecture 1: Introductory Lecture

🎯 Learning Objectives

- To learn the history of C Programming
- To understand the features of the C language
- To develop basic C applications by understanding the structure of a C program

1.1 History of C Programming ⭐

In the 1960s, Dennis Ritchie was working at AT&T Bell Labs to develop an operating system. At that time, all systems used assembly language, which required pages of codes for even simple tasks. The UNIX system, built by both Dennis Ritchie and Ken Thompson, was initially developed using assembly language. Later, they changed the chip system of UNIX from PDA-7 to PDP-11.

During that era, 'B' (developed by Ken Thompson) was the most advanced programming language. They considered implementing it on UNIX, but the compatibility issues of 'B' with PDP-11 led to the development of a new high-level programming language. This new language was later named 'C' because it was the successor to and was heavily influenced by 'B'.

Initially, C was widely used to develop the UNIX operating system. Today, almost all major operating systems are written in C and C++. In 1978, a book named *The C Programming Language* was published, written by Dennis Ritchie and Brian Kernighan, which became widely popular and remains one of the most successful books written about computer science.

Scanned with CamScanner

✅ Ek Baar Padh Lo Bas

Exam mein bas yeh yaad rakhna ki C was created by Dennis Ritchie in 1972 at AT&T Bell Labs, aur yeh 'B' language ka successor hai.

1.2 Why was it named as C? ⭐

This programming language was named 'C' because many of its ideas and principles were derived directly from its predecessor, the programming language named 'B'.

1.3 Why study C? ⭐⭐

The C language was developed for various reasons that made it a very specific and convenient programming language. The C compiler supports both assembly language features and high-level language abstractions. Hence, it is perfectly suited for writing both system applications and most business packages.

It is a portable language, meaning that once the code is written, it can run on any computer system with minimal modifications. C is primarily used for developing Operating Systems. The first Operating System developed entirely using C was UNIX.

The main strength of C lies in its capability to function on any computer architecture along with great flexibility and reliability. Moreover, the vast library of pre-defined functions in C makes it an optimal choice for programming. While assembly language provides extremely powerful programs with maximum control over hardware, it proves to be of great inconvenience when creating large applications due to its lack of portability and readability.

💡 Senior ka Tip

Bro isko aise samjho, Assembly language is like walking on foot—you have full control over every step but going far takes forever. C is like driving a car—it's fast, takes you everywhere (portable), and still lets you steer!

Scanned with CamScanner

1.4 Features of C Language ⭐⭐

C is a procedural programming language. It was mainly developed as a system programming language to write operating systems. The main features of the C language include low-level memory access, a simple set of keywords, and a clean style. These features make the C language highly suitable for system programming, such as operating system or compiler development.

> 🔑 **Definition**
>
> **Procedural Language:** A type of computer programming language that specifies a series of well-structured steps and procedures within its programming context to compose a program. It contains a systematic order of statements, functions, and commands to complete a computational task.

- **Simple and Efficient:** Every C program can be written in English syntax, making it easy to understand and develop. C provides programmers access to direct manipulation with computer hardware, which higher-level languages often restrict. It is fast because statically typed languages are inherently faster than dynamically typed languages.
- **Speed:** Since it is a compiler-based language, it is comparatively faster than interpreter-based programming languages like Java or Python. A compiler translates the entire program as input and generates an executable output file with the object code, whereas an interpreter executes instruction by instruction.
- **Modularity:** The concept of storing C programming language code in the form of libraries for future use is known as modularity. C has its own extensive library to solve common problems, allowing programmers to use specific functions by including header files.
- **Portability:** It is the concept of carrying instructions from one system to another. A C program written and compiled on a Windows operating system can easily run on another Windows-based system. However, a Windows `.exe` file will not execute on a different operating system like Linux.
- **Existence of Libraries:** C comprises a library with a broad range of built-in functions. User-defined functions can also be added to the C library, providing a wide scope for later use and implementation.

Scanned with CamScanner

- **Popular:** It is one of the most widely used languages in the development of operating and embedded systems.
- **Dynamism:** It supports Dynamic Memory Allocation (DMA), which helps in the utilization and management of memory at runtime. The size of a data structure can be altered during runtime using predefined functions like `malloc()`, `calloc()`, `free()`, and `realloc()`.
- **Case Sensitive:** C is a case-sensitive programming language. In C, 'break' and 'BREAK' are treated as two entirely different identifiers.

> ⚠️ **Common Galti**
>
> Yeh galti mat karna! Students aksar 'int' ki jagah 'Int' likh dete hain. C case-sensitive hai, small aur capital letters alag hote hain. Always use lowercase for keywords!

1.5 Applications of C Language ⭐

The applications of C are extensive. Here are some of the most prominent uses:

1. **Operating Systems:** UNIX was designed in C. Later, Microsoft Windows and various Android core applications were scripted in C.
2. **Embedded Systems:** C is an optimum choice for scripting applications and drivers of embedded systems because it is closely related to machine hardware.
3. **GUI (Graphical User Interface):** Software like Adobe Photoshop, Premiere, and Illustrator were created with the help of C.
4. **New Programming Platforms:** C is the base for C++, MATLAB, and Mathematica. It facilitates faster computation.
5. **Google:** The Google file system and Chromium browser were developed using C/C++.
6. **Mozilla Firefox and Thunderbird:** These open-source projects were written in C/C++.
7. **MySQL:** This popular Database Management System was written in C/C++.
8. **Compiler Design:** Several popular compilers were designed using C, such as Bloodshed Dev-C, Clang C, MinGW, and Apple C.

Scanned with CamScanner

9. **Gaming and Animation:** Since C is compiler-based and fast, simple games like Tic-Tac-Toe, Snake, and even advanced graphics engines like Doom3 (developed by id Software in 2004) used C.

1.6 Basic Constructs of a C Program ⭐⭐

Just like English has alphabets, grammar, and sentence structure, the C language has its own building blocks such as variables, keywords, and syntax structures.

The basic building blocks of C are:

- Preprocessor directives
- Header files
- Character set
- Keywords
- Identifiers
- Variables
- Constants
- Operators
- Data types and their storage

1.7 Structure of a C Program ⭐⭐⭐

Let us look at the structure of a standard C program using an example that calculates the power of a number.

Scanned with CamScanner

C PROGRAM

```c
#include <stdio.h>
#include <math.h>

int main() {
    int num, power, result;

    printf("Enter the number: ");
    scanf("%d", &num);

    printf("Enter the power: ");
    scanf("%d", &power);

    if (power < 0) {
        printf("Negative powers are not supported in this simple program.\n");
    } else {
        result = pow(num, power);
        printf("%d to the power %d is %d\n", num, power, result);
    }

    return 0;
}
```

Output:

```
Enter the number: 2
Enter the power: 3
2 to the power 3 is 8
```

Code Explanation:

- `#include <stdio.h>` and `#include <math.h>`: These are preprocessor directives that include the Standard Input/Output library (for printf/scanf) and Math library (for pow function).

Scanned with CamScanner

- `int main()`: The starting point of the program execution. It returns an integer value to the operating system.
- `{ ... }`: The curly braces define the scope or body of the main function.
- `int num, power, result;`: This is a variable declaration statement where we declare variables to hold integer values.
- `printf(...)`: A standard library function used to display output on the console screen.
- `scanf(...)`: A standard library function used to read formatted input from the standard input (keyboard). The `%d` is a format specifier for integers, and `&num` provides the memory address of the variable.
- `if-else`: A conditional control structure to execute code based on a specific condition.
- `return 0;`: This statement indicates that the program has executed successfully without any errors.

1.8 C Character Set and Keywords ⭐

The C language uses the ASCII (American Standard Code for Information Interchange) character set.

- **Alphabets:** Uppercase (A-Z) and Lowercase (a-z). Total 52.
- **Digits:** 0 to 9. Total 10.
- **Special Characters:** Symbols like ~, !, @, #, $, %, ^, &, *, (,), _, +, =, |, \, {, }, [,], :, ;, ", ', <, >, ,, ., ?, /.

> 🔥 **Yeh Exam Mein Zaroor Aayega**
>
> There are exactly 32 keywords in C. Exam mein puchte hain 'How many keywords are there in C?'. Remember the number: 32. Examples: auto, break, case, char, const, continue, default, do, double, else, enum, extern, float, for, goto, if, int, long, register, return, short, signed, sizeof, static, struct, switch, typedef, union, unsigned, void, volatile, while.

Scanned with CamScanner

1.9 Identifiers in C ⭐⭐

Identifiers are the names given to variables, constants, functions, and user-defined data structures.

> 🔑 **Definition**
>
> **Identifier:** A user-defined name given to a programming element like a variable, function, array, or structure to identify it uniquely within a program.

Rules for an Identifier:

1. Must begin with an alphabet or an underscore (_).
2. Must not begin with a digit.
3. They are case-sensitive (e.g., `total` and `Total` are different).
4. Commas or blank spaces are not allowed.
5. Keywords cannot be used as an identifier.
6. Should ideally not exceed 31 characters in length.

Valid: `total`, `sum`, `_x`, `mark_1`

Invalid: `1x` (starts with digit), `char` (keyword), `x+y` (special character '+')

1.10 Compilation Process ⭐⭐⭐

The process of converting C source code into an executable machine code involves several steps:

1. **Source Code:** The `.c` file written by the programmer.
2. **Preprocessor:** Resolves preprocessor directives like `#include` and macros, generating an expanded source code.
3. **Compiler:** Translates the expanded source code into assembly code.

Scanned with CamScanner

4. **Assembler:** Converts the assembly code into object code (`.obj` or `.o`).
5. **Linker:** Links the object code with library files and other object modules to generate a final executable file (`.exe` or `a.out`).

📌 Quick Revision — Lecture 1

- C was created by Dennis Ritchie in 1972 at Bell Labs.
- C is a middle-level, procedural, and structured language.
- Main features: Portable, fast, modular, dynamic memory allocation.
- Structure: Preprocessor directives -> main() function -> variable declaration -> body -> return statement.
- Keywords are reserved words (32 in C); Identifiers are user-defined names.
- Compilation process involves: Preprocessor -> Compiler -> Assembler -> Linker.

Scanned with CamScanner

📝 Practice Questions — Lecture 1

MCQ Q1

What is the correct value to return to the operating system upon the successful completion of a program?

A) 1
B) -1
C) 0
D) Programs do not return a value.

> **Correct Answer: C) 0**
> *Explanation:* In C, returning 0 from the main function signifies that the program has executed successfully without any errors.

MCQ Q2

Which of the following is an invalid identifier in C?

A) _myVar
B) myVar_1
C) 1stVar
D) Var1

> **Correct Answer: C) 1stVar**
> *Explanation:* An identifier in C cannot start with a digit. Therefore, 1stVar is invalid.

Scanned with CamScanner

MCQ Q3

The return type of main function is double. (True / False)

A) True

B) False

> **Correct Answer: B) False**
> *Explanation:* The standard return type of the main function is 'int'.

Short Answer Q4

Why is C called a procedural language?

> **Answer:** C is called a procedural language because its programs follow a step-by-step procedure of written instructions called functions. It follows a top-down approach where importance is given to the flow of control and functions rather than the data on which the functions operate.

Short Answer Q5

What is a preprocessor in C?

> **Answer:** A preprocessor is a program that processes the source code before it passes through the compiler. It handles directives that begin with a '#' symbol, such as '#include' for file inclusion and '#define' for macros.

Scanned with CamScanner

CU Helpdesk — Your Academic Support Community

Click Here to Join WhatsApp Group

02 Lecture 2: Introduction to Flowcharts

🎯 Learning Objectives

- To learn the basic symbols used in flowcharts
- To learn how to create flowcharts for different problems and processes

2.1 What is a Flowchart? ⭐

🔑 Definition

Flowchart: A graphical representation of an algorithm or process. It illustrates steps in a sequential order using boxes of various kinds, connected by arrows to represent the flow of control.

Programmers often use flowcharts as a program-planning tool to solve complex problems visually before writing any actual code. The systematic process of drawing a flowchart for an algorithm is referred to as "flowcharting".

Scanned with CamScanner

2.2 Use of a Flowchart ⭐

Flowcharts are immensely popular worldwide for process documentation. They are primarily used to:

- Improve processes by identifying bottlenecks, unnecessary steps, and defects.
- Communicate and explain a complicated process to team members or non-technical stakeholders clearly.
- Easily visualize complex workflows and system architectures.

2.3 Understanding Flowchart Symbols ⭐⭐

To draw standard flowcharts, specific geometric shapes are used to represent different types of instructions or events.

- **Terminal / Terminator (Oval):** Indicates the beginning (Start) or ending (End) point of a flowchart.
- **Process (Rectangle):** Represents a process, action step, or mathematical operation (e.g., `sum = a + b`).
- **Data / I/O (Parallelogram):** Represents Input and Output operations (e.g., `Read a, b` or `Print sum`).
- **Decision (Diamond):** Used to represent a decision point. It contains a condition that results in a 'Yes' or 'No' (True or False) branching path.
- **Document:** A rectangle with a wave-like base, representing a document or report generated in the process.
- **Arrows:** Indicate the direction of the flow of logic.
- **Subroutine (Rectangle with double vertical edges):** Represents a predefined process or a function call to another flowchart/module.
- **Circle:** Used as an on-page connector to join different parts of a complex flowchart on the same page.

Scanned with CamScanner

> ⚠️ **Common Galti**
>
> Bachhe aksar Diamond (Decision) box se sirf ek line nikalte hain. Yaad rakhna, Decision box ke humesha TWO outputs hone chahiye—ek True/Yes ke liye, dusra False/No ke liye!

2.4 Flowchart Examples ⭐⭐⭐

Example 1: Flowchart to find the sum of two numbers

Steps involved:

1. Start the process.
2. Read two numbers, `A` and `B` as input.
3. Calculate the sum: `Sum = A + B`.
4. Display the calculated `Sum`.
5. End the process.

> 💡 **Senior ka Tip**
>
> Flowchart draw karte time symbols ka dhyaan rakho. Start/End hamesha Oval mein hoga. Input/Output hamesha Parallelogram mein, aur calculation (jahan bhi equal '=' sign aaye) Rectangle mein hoga.

Example 2: Flowchart to calculate Profit or Loss

Steps involved:

1. Start.
2. Read Income and Costs.
3. Decision: Is Income greater than Costs?
4. If Yes: Calculate Profit = Income - Costs. Print Profit.
5. If No: Calculate Loss = Costs - Income. Print Loss.

Scanned with CamScanner

6. End.

📌 Quick Revision — Lecture 2

- A flowchart is a visual diagram of an algorithm.
- Oval: Start/Stop.
- Rectangle: Process/Calculation.
- Parallelogram: Input/Output.
- Diamond: Decision making (Yes/No).
- Arrows: Direction of flow.

📝 Practice Questions — Lecture 2

MCQ Q1

Which symbol is used to represent a decision in a flowchart?

A) Rectangle
B) Oval
C) Parallelogram
D) Diamond

> **Correct Answer: D) Diamond**
> *Explanation:* The diamond shape is used for conditions, yielding True or False paths.

Scanned with CamScanner

MCQ Q2

The parallelogram shape in a flowchart represents:

A) Start/Stop

B) Process

C) Input/Output

D) Connector

> **Correct Answer: C) Input/Output**
> *Explanation:* Parallelograms indicate data entering the system (input) or leaving the system (output).

Short Answer Q3

What is a sub-process in a flowchart?

> **Answer:** Complex processes are often broken down into smaller, manageable sub-processes for clarity. A flowchart can reference a different sub-process using the predefined process symbol (a rectangle with double vertical edges).

Short Answer Q4

How are flowcharts used in computer programming?

> **Answer:** Flowcharts are used to visually map out algorithms and processes before writing code. They help programmers comprehend the logic, plan the software architecture, and communicate the program flow to non-technical stakeholders.

Scanned with CamScanner

Long Answer Q5

Write the textual steps (Algorithm) to accompany a flowchart that checks if a number is Even or Odd.

Answer:

1. Start
2. Input number 'N'
3. If N % 2 == 0, then go to Step 4, else go to Step 5
4. Print "Number is Even". Go to Step 6.
5. Print "Number is Odd".
6. End

Scanned with CamScanner

CU Helpdesk — Your Academic Support Community

Click Here to Join WhatsApp Group

03 Lecture 3: Introduction to Algorithms

🎯 Learning Objectives

- To understand the absolute importance of algorithms in programming
- To learn the structured approach to write algorithms
- To differentiate and evaluate different algorithms designed for the same task

3.1 Introduction to Algorithms ⭐

🔑 Definition

Algorithm: A finite, step-by-step sequence of well-defined instructions designed to solve a specific problem or perform a computation, producing an outcome within a finite amount of time.

When tasks involve a large amount of input data, complex manipulation, or both, programmers need to construct clever and efficient algorithms that a computer can execute quickly.

The best algorithms possess specific characteristics: they run fast, use memory and other computer resources sparingly, and are easy to read and modify. Therefore, the study of algorithms is fundamental to computer science and software engineering.

Scanned with CamScanner

Algorithms are not exclusive to computers. For example, leaving precise, conditional instructions for a pet sitter ("If the bowl is empty, add 5 pellets; otherwise, add 3 pellets") is an everyday application of an algorithm.

3.2 Conditional Instructions in Algorithms ⭐⭐

Algorithms in computer science are rarely just linear sequences of commands. They frequently include conditional instructions. This pattern generally follows three steps:

- Describe a specific test or condition.
- List instructions to execute if the test succeeds (True).
- List alternative instructions to execute if the test fails (False).

Nearly every modern programming language uses the structural keywords `if` and `else` for implementing these conditional paths.

3.3 How to Design an Algorithm? ⭐⭐

Before writing an algorithm, the following prerequisites must be clearly understood:

1. **The Problem:** The exact nature of the problem to be solved.
2. **Constraints:** The limitations or rules that must be adhered to while solving the problem.
3. **Inputs:** The data required to solve the problem.
4. **Outputs:** The expected result upon solving the problem.
5. **The Solution:** A clear logical pathway bridging the inputs to the outputs under the given constraints.

Example: Algorithm to Add 3 Numbers

1. START
2. Declare three integer variables: `num1`, `num2`, and `num3`.
3. Take the three numbers as inputs into the respective variables.
4. Declare an integer variable `sum`.

Scanned with CamScanner

5. Add the three numbers and store the result: `sum = num1 + num2 + num3`.
6. Print the value of `sum`.
7. END

> **💡 Senior ka Tip**
>
> Bro, algorithm is literally just plain English version of your code. Hamesha START se shuru karo, input lo, process karo, print karo, aur END pe khatam. Step numbering zaroori hai!

3.4 Analyzing the Performance of an Algorithm ⭐⭐⭐

A single problem can be solved in multiple ways, yielding different algorithms. The next step is to evaluate these proposed solutions and select the optimal one. Performance analysis measures depend on:

- Accuracy: Does it provide the exact, correct solution?
- Readability: Is it easy to understand and maintain?
- Simplicity: Is it easy to implement in code?
- **Space Complexity:** How much memory (RAM) it requires during execution.
- **Time Complexity:** How much time (processing cycles) it takes to complete.

In formal computer science, when analyzing an algorithm, we predominantly focus on **Time Complexity** and **Space Complexity**.

> **🔥 Yeh Exam Mein Zaroor Aayega**
>
> Exam mein "Time and Space Complexity" par question pakka aata hai. Remember: Time complexity is about Execution Time, and Space complexity is about Memory usage. The best algorithm takes minimum time and minimum space.

Scanned with CamScanner

3.5 Example: Finding the Maximum of Three Numbers ⭐⭐⭐

Algorithm:

1. START
2. Input three numbers `a`, `b`, `c`
3. If `a > b`, go to step 4, otherwise go to step 5
4. If `a > c`, go to step 6, otherwise go to step 8
5. If `b > c`, go to step 7, otherwise go to step 8
6. Output "a is the largest", go to step 9
7. Output "b is the largest", go to step 9
8. Output "c is the largest", go to step 9
9. STOP

Scanned with CamScanner

DRY RUN TABLE

Inputs (a, b, c)	Step 3 (a > b)	Step 4 (a > c)	Step 5 (b > c)	Output
5, 20, 8	5 > 20 (False) -> Step 5	-	20 > 8 (True) -> Step 7	"b is the largest"

Scanned with CamScanner

📌 Quick Revision — Lecture 3

- Algorithm: Step-by-step logic to solve a problem.
- Algorithms utilize sequences, conditionals (if/else), and loops.
- Design prerequisites: Problem, Constraints, Inputs, Outputs, Solution logic.
- Performance Analysis is based mainly on Time Complexity and Space Complexity.
- Time complexity: Execution speed. Space complexity: Memory consumed.

📝 Practice Questions — Lecture 3

MCQ Q1

Which of the following is NOT a fundamental metric for formally analyzing an algorithm's efficiency?

A) Time Complexity
B) Space Complexity
C) Number of lines of code
D) Memory consumed

> **Correct Answer: C) Number of lines of code**
> *Explanation:* The number of lines is not a reliable metric. Formal analysis relies on Time and Space complexities.

Scanned with CamScanner

MCQ Q2

What does "Space Complexity" of an algorithm refer to?

A) Hard disk space

B) Memory space (RAM) required during execution

C) Cloud storage space

D) Code file size

> **Correct Answer: B) Memory space (RAM) required during execution**
> *Explanation:* Space complexity measures the total amount of runtime memory needed by an algorithm to run completely.

Short Answer Q3

Define an algorithm in computer science.

> **Answer:** An algorithm is a finite, ordered sequence of unambiguous instructions designed to perform a specific task or solve a computational problem.

Short Answer Q4

Name three different types of algorithm design techniques.

> **Answer:** Divide and conquer algorithm, Greedy algorithm, and Dynamic programming algorithm.

Scanned with CamScanner

Write Program / Algorithm Q5

Write an algorithm to calculate the area of a rectangle.

Answer:

1. START
2. Declare variables `length`, `width`, and `area`.
3. Input the values of `length` and `width`.
4. Calculate area using the formula: `area = length * width`.
5. Print the calculated `area`.
6. END

Scanned with CamScanner

CU Helpdesk — Your Academic Support Community

Click Here to Join WhatsApp Group

04 Lecture 4: Memory Layout of C Program

🎯 Learning Objectives

- To comprehend the memory layout of C Programs and how they are stored in RAM.
- To understand different memory segments like Text, Data, Stack, and Heap.

4.1 Introduction to Memory Layout ⭐

A C language program is designed so that the programmer can decide the amount of memory to use. When a program is executed, its binary executable image is loaded into the RAM in a highly organized manner. A standard memory layout of a C program comprises five primary sections:

- Text segment (Code segment)
- Initialized data segment
- Uninitialized data segment (BSS)
- Stack
- Heap

Scanned with CamScanner

✅ Ek Baar Padh Lo Bas

Har ek section ka apna alag kaam hota hai. Code kahin aur store hota hai, variables kahin aur, aur temporary data alag jagah. Bas inka naam aur kaam yaad rakhna hai.

4.2 Text Segment / Code Segment ⭐⭐

When we compile a program, a binary file is generated (for example, `a.out` or `program.exe`). This file consists of machine-level instructions. These instructions are stored in the text segment.

The text segment is a shared and read-only memory region. It is read-only to prevent a program from being accidentally modified during execution. It is shared so that multiple instances of the same application can run simultaneously using the same code base without consuming extra memory.

🔑 Definition

Text Segment: Also known as the code segment, it contains the executable machine code of the compiled program. It is typically read-only and shared.

4.3 Data Segments ⭐⭐

The data segment stores the program's data. This includes both initialized and uninitialized variables. This segment is sub-divided into two main parts:

4.3.1 Initialized Data Segment

This segment stores all global, static, constant, and external variables that are explicitly initialized with a value by the programmer in the source code. Unlike the text segment, it is not read-only (unless variables are declared as constant), meaning the values of the variables can be altered at runtime.

Scanned with CamScanner

The values of these variables are read directly from the executable file when the program is loaded into memory.

4.3.2 Uninitialized Data Segment (BSS)

This segment contains global and static variables that are not explicitly initialized in the source code. The system automatically initializes all memory in this segment to ZERO (0) before the program starts executing.

This segment is also known as BSS (Block Started by Symbol). The actual space for this segment is allocated by the loader at run-time, so the executable file merely needs to record its location and size, reducing the binary file size.

> **💡 Senior ka Tip**
>
> Bro isko aise samjho: Agar tumne variable ko value de di (jaise `int a = 10;`), toh wo "Initialized" wale room mein jayega. Agar sirf declare karke chhod diya (jaise `int a;`), toh compiler usko default '0' dekar "BSS" wale room mein bhej dega.

Scanned with CamScanner

C PROGRAM

```c
#include <stdio.h>

int global_var = 10; // Stored in Initialized Data Segment
int uninit_global_var; // Stored in Uninitialized Data Segment (BSS),
initialized to 0

int main() {
    static int static_var = 5; // Stored in Initialized Data Segment
    static int uninit_static; // Stored in Uninitialized Data Segment (BSS)

    printf("Global initialized: %d\n", global_var);
    printf("Global uninitialized: %d\n", uninit_global_var);

    return 0;
}
```

📺 **Output:**

```
Global initialized: 10 Global uninitialized: 0
```

4.4 Stack Segment ⭐⭐⭐

A stack is a conceptual data structure based on the Last In First Out (LIFO) principle. It is primarily used for static memory allocation and managing function calls.

Every time a function is called, a new "frame" is pushed onto the stack. This frame stores the address of where to return after the function finishes, information about the caller's environment, machine registers, and all the automatic (local) and temporary variables defined within that function.

Scanned with CamScanner

When the function completes its execution, its frame is popped from the stack, and the memory is freed automatically. The stack pointer register tracks the top of the stack.

> ⚠️ **Common Galti**
>
> Local variables ka scope sirf function ke andar hota hai kyunki unka data stack par hota hai. Function khatam hote hi stack clear ho jata hai. Galti se local variable ka address return mat karna!

4.5 Heap Segment ⭐⭐⭐

The heap is the segment where dynamic memory allocation takes place. If you do not know how much memory you need at compile time, you can allocate it dynamically at runtime using functions like `malloc()`, `calloc()`, and `realloc()`. Memory allocated here must be manually released using the `free()` function.

In the memory layout, the stack and heap segments typically grow in opposite directions. The stack grows downwards from a higher address, while the heap grows upwards. When the stack pointer meets the heap pointer, the system runs out of free memory.

> 🔥 **Yeh Exam Mein Zaroor Aayega**
>
> Difference between Stack and Heap is a very common exam question.

Scanned with CamScanner

Feature	Stack Segment	Heap Segment
Memory Allocation	Static (determined at compile-time)	Dynamic (requested at run-time)
Management	Automatic (managed by the CPU)	Manual (managed by programmer using malloc/free)
Access Speed	Very fast	Slower compared to Stack
Data Storage	Local variables, function return addresses	Dynamically created variables and large datasets

📌 Quick Revision — Lecture 4

- **Text Segment:** Stores compiled machine code. Read-only.
- **Initialized Data:** Stores global/static variables with initial values.
- **Uninitialized Data (BSS):** Stores global/static variables without initial values (defaults to 0).
- **Stack:** Stores local variables and manages function calls (LIFO). Automatic allocation.
- **Heap:** Used for dynamic memory allocation (e.g., using malloc). Manual allocation.

Scanned with CamScanner

📝 Practice Questions — Lecture 4

MCQ 1

Which memory segment is read-only?

A) Stack

B) Heap

C) Text Segment

D) BSS Segment

> **Correct Answer: C) Text Segment**
>
> *Explanation:* The text segment contains the compiled machine code instructions and is kept read-only to prevent accidental modifications during execution.

MCQ 2

What is the default value of variables stored in the uninitialized data segment (BSS)?

A) Garbage value

B) 1

C) -1

D) 0

> **Correct Answer: D) 0**
>
> *Explanation:* The system automatically initializes all memory in the BSS segment to arithmetic zero before the program starts execution.

Scanned with CamScanner

Short Answer 3

Identify where the variable `static int count;` will be stored in memory.

> **Answer:** It will be stored in the Uninitialized Data Segment (BSS) because it is a static variable without explicit initialization.

Short Answer 4

Why do the stack and heap grow in opposite directions?

> **Answer:** Growing in opposite directions maximizes the utilization of the available free memory space between them. They share the same available memory region, and when they meet, the free memory is exhausted.

Code Output 5

Where is the variable `c` stored in this code? `char c[] = "student";` declared globally?

> **Answer:** It is a globally declared variable that is initialized, so it is stored in the Initialized Data Segment.

Scanned with CamScanner

CU Helpdesk — Your Academic Support Community

Click Here to Join WhatsApp Group

05 Lecture 5: Input Output Functions in C

🎯 Learning Objectives

- To learn different functions available for input and output operations.
- To differentiate between formatted and unformatted I/O functions.
- To understand the importance of indentation and comments in programming.

5.1 Introduction to Input and Output ⭐

In order to read data from standard input devices (like a keyboard) and display data on standard output devices (like a monitor), C provides a collection of built-in functions. These functions handle the transfer of data and are categorized into two types:

- **Formatted I/O:** Allows formatting the data (like controlling decimal places or padding) using format specifiers.
- **Unformatted I/O:** Works strictly with character data types and treats input/output as a simple stream of characters.

5.2 Unformatted I/O Functions ⭐⭐

Unformatted input and output functions only work with character data types. They do not require any format specifiers (like `%d` or `%c`).

Scanned with CamScanner

Function	Description
`getch()`	Reads a single character from the console without displaying (echoing) it. Returns immediately without waiting for the Enter key.
`getche()`	Reads a single character from the console and displays (echoes) it. Returns immediately without waiting for the Enter key.
`getchar()`	Reads a single character, displays it, but waits for the Enter key to be pressed before returning.
`putchar()`	Prints a single character to the standard output screen.
`gets()`	Reads a complete string entered by the user, including spaces, until the Enter key is pressed.
`puts()`	Prints a string to the standard output and automatically appends a newline.

💡 Senior ka Tip

Bro, jab sirf Yes/No (Y/N) option lena ho toh `getche()` best hai kyunki user ko Enter press karne ki zaroorat nahi padti. Agar user se pura naam space ke saath lena ho (jaise "John Doe"), toh `scanf()` kaam nahi aayega, tab `gets()` use karna padega.

Scanned with CamScanner

C PROGRAM

```c
#include <stdio.h>
int main() {
    char name[25];
    printf("Enter your Full Name: ");
    gets(name); // Reads string including spaces
    printf("Welcome, ");
    puts(name); // Prints string
    return 0;
}
```

📺 **Output:**

```
Enter your Full Name: John Doe Welcome, John Doe
```

5.3 Formatted I/O Functions ⭐⭐

Formatted I/O functions use format specifiers to determine how the input or output should be structured. The two most fundamental formatted functions are `printf()` and `scanf()`.

🔑 Definition

Format Specifier: A placeholder used in formatted I/O functions (like %d, %f, %c) that tells the compiler what type of data to read or print.

Scanned with CamScanner

Format Specifier	Description
`%d`	Signed Integer
`%u`	Unsigned Integer
`%c`	Single Character
`%f`	Float (real value)
`%lf`	Double (long real value)
`%s`	String (Character array)

> ⚠️ **Common Galti**
>
> Hamesha yaad rakhna ki `scanf()` mein variables ke aage `&` (ampersand) lagana zaroori hai (e.g., `scanf("%d", &a);`), kyunki humein variable ka memory address pass karna hota hai. Lekin `printf()` mein `&` nahi lagate! Strings (`%s`) ke case mein `&` ki zaroorat nahi hoti.

5.3.1 Formatting Output with Precision

You can control exactly how output is printed, such as specifying the width or limiting the decimal places for real numbers.

Scanned with CamScanner

C PROGRAM

```c
#include <stdio.h>
int main() {
    float a = 987.321;
    printf("/%f/\n", a);      // Default precision is 6 decimal places
    printf("/%12f/\n", a);    // Width of 12 spaces, right aligned
    printf("/%-12f/\n", a);   // Width of 12 spaces, left aligned
    printf("/%.2f/\n", a);    // Precision of 2 decimal places
    return 0;
}
```

📺 **Output:**

```
/987.321000/ / 987.321000/ /987.321000 / /987.32/
```

5.4 Indentation and Comments ⭐

Indentation: Indentation involves adding blank spaces or tabs at the beginning of lines of code. It is not mandatory for the compiler, but it drastically improves code readability for humans. It helps visually structure loops, conditions, and functions.

Comments: Comments are explanations written within the source code that are completely ignored by the compiler. They help developers understand the logic.

- **Single Line Comment:** Starts with `//`. Everything following it on the same line is ignored.
- **Multi-Line Comment:** Enclosed between `/*` and `*/`. Can span multiple lines.

5.5 Header Files ⭐⭐

Header files contain the declarations of functions, macros, and variables. Every C program must include at least one header file, typically `<stdio.h>`, which contains declarations for standard I/O

Scanned with CamScanner

functions like `printf()` and `scanf()`.

- **Predefined Header Files:** Provided by the C system, included using angle brackets, e.g., `#include <stdio.h>` or `#include <math.h>`.
- **User-defined Header Files:** Created by the programmer, included using double quotes, e.g., `#include "myheader.h"`.

📌 Quick Revision — Lecture 5

- **Formatted I/O:** `printf()` and `scanf()`, requires format specifiers (%d, %f, %c, %s).
- **Unformatted I/O:** Works with characters only. `getch()` (no echo), `getche()` (echo), `getchar()` (requires Enter), `gets()` (reads string with spaces), `puts()`.
- **Format Precision:** Can format decimal points using `%.2f` inside printf.
- **Indentation & Comments:** Enhances code readability. Compiler ignores comments (`//` or `/* ... */`).
- **Header Files:** `<math.h>`, `<stdio.h>` are predefined. `"file.h"` is user-defined.

Scanned with CamScanner

📝 Practice Questions — Lecture 5

MCQ 1

Which function is used to read a string including spaces?

A) scanf()
B) gets()
C) getchar()
D) getche()

> **Correct Answer: B) gets()**
> *Explanation:* The `gets()` function reads characters until a newline is encountered, allowing it to read strings containing spaces, unlike `scanf()` with the `%s` specifier.

MCQ 2

What format specifier is used for variables of type `double`?

A) %d
B) %f
C) %lf
D) %ld

> **Correct Answer: C) %lf**
> *Explanation:* The format specifier `%lf` stands for long float, which corresponds to the double data type.

Scanned with CamScanner

Find the Error 3

What is wrong with this statement: `scanf("%d %d", a, b);`

> **Answer:** The ampersand `&` is missing before the variables. It should be `scanf("%d %d", &a, &b);` because `scanf()` requires the memory addresses of the variables to store the incoming data.

Code Output 4

Predict the output:

```
int a = 367;
printf("*%05d*", a);
```

> **Answer:** `*00367*`
> *Explanation:* The specifier `%05d` dictates a minimum width of 5 characters, and empty spaces should be padded with zeros.

Short Answer 5

Explain the difference between `#include <header.h>` and `#include "header.h"`.

> **Answer:** Angle brackets `< >` are used for standard system header files (predefined). Double quotes `" "` are used for user-defined header files, and the compiler searches for them in the current directory first.

Scanned with CamScanner

CU Helpdesk — Your Academic Support Community

Click Here to Join WhatsApp Group

06 Lecture 6: Data Types, Variables, and Constants

🎯 Learning Objectives

- To learn how to declare different types of variables based on program requirements.
- To understand different data types, their ranges, and memory spaces.
- To differentiate between variables and constants.

6.1 Data Types in C ⭐

A data type specifies the type of data that a variable can hold and defines the amount of memory space allocated for it. Every variable in C must be associated with a specific data type. Data types in C are categorized into three broad categories:

- **Primary (Predefined) Data Types:** Built directly into the C compiler. Examples: `int`, `char`, `float`, `double`.
- **Derived Data Types:** Created by combining primary data types. Examples: Arrays, Pointers.
- **User-defined Data Types:** Defined by the user to create complex structures or aliases. Examples: `struct`, `union`, `typedef`.

Scanned with CamScanner

6.1.1 Primary Data Types Memory and Ranges

Each primary data type has a fixed memory size and range. Note that sizes can depend on compiler architecture (e.g., 16-bit vs 32-bit).

Data Type	Memory Space (typical)	Format Specifier
`char` (signed)	1 Byte	`%c`
`int` (signed)	2 Bytes (or 4 Bytes)	`%d`
`long` (signed)	4 Bytes	`%ld`
`float`	4 Bytes	`%f`
`double`	8 Bytes	`%lf`

> ✅ **Ek Baar Padh Lo Bas**
>
> Range yaad rakhna mushkil lag sakta hai. Bas memory sizes yaad rakho: char hamesha 1 byte leta hai. float 4 bytes aur double 8 bytes leta hai. integer compiler par depend karta hai, usually 2 ya 4 bytes.

6.2 Constants ⭐⭐

Constants are fixed values that do not change during the execution of a program. They are categorized into several types:

- **Integer Constants:** Whole numbers. Can be decimal (base 10), octal (base 8, prefixed with `0`), or hexadecimal (base 16, prefixed with `0x` or `0X`).
- **Floating-point Constants:** Real numbers containing a decimal point or an exponential part (e.g., `3.14`, `341E-5`).

Scanned with CamScanner

- **Character Constants:** A single character enclosed in single quotes (e.g., `'A'`, `'5'`). Special escape sequences like `'\n'` (newline) or `'\t'` (tab) are also valid character constants.
- **String Constants:** A sequence of characters enclosed in double quotes (e.g., `"Hello World"`).

Defining Constants

Constants can be explicitly defined in code using two methods:

1. Using the `const` keyword: `const double PI = 3.142;`
2. Using macros with `#define`: `#define PI 3.142` (Note: no equals sign or semicolon).

6.3 Variables ⭐⭐

Variables are named memory locations whose values can change during program execution. Before using a variable, it must be declared with a specific data type.

6.3.1 Rules for Naming Variables (Identifiers)

- Must consist only of alphabets, digits, and underscores.
- The first character cannot be a digit (e.g., `1stPlace` is invalid).
- No special characters or spaces are allowed (e.g., `my var` or `my@var` are invalid).
- Keywords (like `if`, `while`, `int`) cannot be used as variable names.

> ⚠️ **Common Galti**
>
> C case-sensitive language hai. Iska matlab `int Total;` aur `int total;` dono alag variables hain. Galti se capital aur small letters mix mat karna!

6.3.2 Local vs Global Variables ⭐⭐⭐

Variables are defined by their scope (where they can be accessed) and lifetime (how long they stay in memory).

Scanned with CamScanner

- **Local Variable:** Declared inside a function or block. It can only be accessed within that specific block. Its lifetime ends as soon as the function finishes execution.
- **Global Variable:** Declared outside all functions. It can be accessed by any function in the program. Its lifetime spans the entire execution of the program.

C PROGRAM

```c
#include <stdio.h>

int global_var = 100; // Global Variable

void myFunction() {
    int local_var = 50; // Local Variable
    printf("Inside function - Global: %d, Local: %d\n", global_var, local_var);
}

int main() {
    myFunction();
    printf("Inside main - Global: %d\n", global_var);
    // printf("%d", local_var); // ERROR! local_var is not accessible here
    return 0;
}
```

📺 **Output:**

```
Inside function - Global: 100, Local: 50 Inside main - Global: 100
```

6.4 L-values and R-values ⭐⭐

In C expressions involving the assignment operator (`=`):

- **lvalue (Locator Value):** Refers to an expression that has a specific memory location. It can appear on either the left or right side of an assignment operator. Variables are lvalues.

Scanned with CamScanner

- **rvalue:** Refers to a data value or constant that is stored at some address in memory. It can ONLY appear on the right side of the assignment operator.

Example: `a = 10;` is valid (variable `a` is an lvalue, `10` is an rvalue). However, `10 = a;` is invalid because an rvalue cannot be assigned a value.

📌 Quick Revision — Lecture 6

- **Primary Data Types:** `int`, `char`, `float`, `double`.
- **Constants:** Values that never change. Defined using `const` or `#define`.
- **Variables:** Named memory spaces that hold data. Cannot start with numbers or contain special symbols.
- **Scope:** Local variables exist only in their function; global variables exist throughout the program.
- **L-value vs R-value:** L-value represents a memory address (can be on LHS); R-value represents a pure data value (only RHS).

Scanned with CamScanner

📝 Practice Questions — Lecture 6

MCQ 1

Which of the following is an invalid variable name?

A) _myVar

B) my_var_123

C) 1st_variable

D) total_sum

> **Correct Answer: C) 1st_variable**
>
> *Explanation:* A variable identifier cannot begin with a numerical digit.

MCQ 2

What is the output of the following code?

```
int a = 50;
float a = 50.0;
printf("%d", a);
```

A) 50

B) 50.0000

C) Compilation Error

D) Garbage Value

> **Correct Answer: C) Compilation Error**
>
> *Explanation:* A variable identifier must be unique within its scope. You cannot declare a variable named `a` twice in the same scope, even with different data types.

Scanned with CamScanner

Short Answer 3

Explain the difference between a character constant and a string constant. Provide examples.

> **Answer:** A character constant represents a single character enclosed in single quotes (e.g., `'A'`) and takes 1 byte. A string constant represents a sequence of characters enclosed in double quotes (e.g., `"Hello"`) and includes a hidden null terminator at the end.

Short Answer 4

What is the significance of the `typedef` keyword?

> **Answer:** The `typedef` keyword is used to create user-defined data types by providing an alternative name or alias for an existing primary or derived data type. It makes the code more readable and easier to manage.

Find the Error 5

Identify the error in this code:

```
const int max = 100;
max = 150;
```

> **Answer:** The variable `max` is declared as a `const` (constant). This means its value cannot be modified after initialization. Attempting to assign `150` to it will result in a compilation error.

Scanned with CamScanner

CU Helpdesk — Your Academic Support Community

Click Here to Join WhatsApp Group

07 Lecture 7: Introduction to Operators in C

🎯 Learning Objectives

- To learn different types of operators in the C programming language and understand their working.
- To evaluate mathematical and logical expressions using these operators.

7.1 Introduction ⭐

A large variety of built-in operators are supported by the C programming language. An operator can be explained as a symbol that instructs the compiler to perform certain mathematical or logical manipulations. Operators act upon data items, known as operands, to produce a specific result.

🔑 Definition

An **operator** is a symbol that operates on a value or a variable. The data items that operators act upon are called **operands**.

Scanned with CamScanner

> **💡 Senior ka Tip**
>
> Bro isko aise samjho: Agar tum `A + B` likhte ho, toh `+` operator hai jo compiler ko bolta hai ki dono ko add karo, aur `A` aur `B` operands hain. Basic math hi hai!

7.2 Types of Operators ⭐⭐

C provides a rich set of operators, which can be broadly classified into 8 categories:

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Assignment Operators
- Increment and Decrement Operators
- Conditional (Ternary) Operators
- Bitwise Operators (Covered in Lecture 8)
- Miscellaneous Operators (sizeof, comma, etc.)

7.2.1 Arithmetic Operators

Arithmetic operators are used to perform mathematical operations such as addition, subtraction, multiplication, and division. C provides the following arithmetic operators: `+`, `-`, `*`, `/`, `%`.

Scanned with CamScanner

Operator	Meaning	Example (A=10, B=20)
+	Adds two operands	A+B = 30
-	Subtracts second operand from the first	A-B = -10
*	Multiplies two operands	A*B = 200
/	Divides numerator by denominator	A/B = 0
%	Remainder of division (Modulus)	A%B = 10

⚠️ Common Galti

Modulus (`%`) operator float values pe kaam nahi karta! Agar tum `3.5 % 2` likhoge toh compiler error dega "Illegal use of Floating Point". Isko sirf integers pe use karna.

Scanned with CamScanner

C PROGRAM

```c
#include <stdio.h>
int main() {
    int a, b;
    printf("Enter the value of a and b: ");
    scanf("%d %d", &a, &b);  // Input the values of a and b

    printf("Result of a+b is %d\n", a + b); // Addition
    printf("Result of a-b is %d\n", a - b); // Subtraction
    printf("Result of a*b is %d\n", a * b); // Multiplication
    printf("Result of a/b is %d\n", a / b); // Division
    printf("Result of a%%b is %d\n", a % b); // Remainder

    return 0;
}
```

📺 **Output:**

```
 Enter the value of a and b: 10 20
Result of a+b is 30
Result of a-b is -10
Result of a*b is 200
Result of a/b is 0
Result of a%b is 10
```

7.2.2 Relational Operators

A relational operator checks the relationship between two operands. If the relation is true, it returns `1`; if the relation is false, it returns `0`.

Scanned with CamScanner

Operator	Meaning	Example
>	Greater than	2 > 4 is 0 (False)
<	Less than	2 < 4 is 1 (True)
>=	Greater than or equal to	2 >= 4 is 0 (False)
<=	Less than or equal to	2 <= 4 is 1 (True)
==	Equal to	2 == 4 is 0 (False)
!=	Not equal to	2 != 4 is 1 (True)

7.2.3 Logical Operators

An expression containing a logical operator returns either 0 (false) or 1 (true) depending upon whether the logical expression results in true or false.

Operator	Meaning	Example (c=5, d=2)
&& (AND)	True only if all operands are true.	(c==5) && (d>5) is 0
|| (OR)	True if at least one operand is true.	(c==5) || (d>5) is 1
! (NOT)	True only if the operand is 0 (false).	!(c==5) is 0

✅ Ek Baar Padh Lo Bas

Logical AND (&&) mein dono conditions true honi chahiye. Logical OR (||) mein koi ek true ho jaye toh kaam chal jayega. NOT (!) seedha ulta kar deta hai (True ko False, False ko True).

Scanned with CamScanner

7.2.4 Assignment Operator

Assignment operators are used to assign the result of an expression to a variable. The basic assignment operator is `=`.

Operator	Example	Same as
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y

7.2.5 Increment and Decrement Operators

C programming has two unary operators: increment `++` and decrement `--`, to change the value of an operand by 1.

- **Pre-increment/decrement:** `++a` or `--b`. The value is changed first, and then the expression is evaluated.
- **Post-increment/decrement:** `a++` or `b--`. The expression is evaluated with the original value, and then it is changed.

> **🔥 Yeh Exam Mein Zaroor Aayega**
>
> Exam mein post-increment aur pre-increment ka difference output questions mein zaroor puchte hain! Dhyan rakhna ki `y = x++` mein y ko purani value milegi.

Scanned with CamScanner

7.2.6 Conditional (Ternary) Operator

The conditional operator evaluates an expression based on a condition, working as a shorthand for the `if-else` statement.

Syntax: `expression1 ? expression2 : expression3;`

If expression1 is true, expression2 executes. Otherwise, expression3 executes.

7.2.7 Miscellaneous Operators (sizeof, comma)

sizeof Operator: Calculates the memory size (in bytes) of a variable or a data type.

Comma Operator (,): Used to combine two or more expressions. When used as an operator (and not a separator), it evaluates expressions from left to right and returns the value of the rightmost operand.

📌 Quick Revision — Lecture 7

- Operators manipulate operands to produce results.
- Modulus (%) yields the remainder but only works with integers.
- Relational and Logical operators return 1 (True) or 0 (False).
- Ternary operator is a compact alternative to if-else conditionals.
- The comma operator evaluates all sub-expressions but assigns the rightmost value.

Scanned with CamScanner

📝 Practice Questions — Lecture 7

Q1

Which operator is used to find the remainder of a division in C?

A) /

B) %

C) //

D) MOD

> **Correct Answer: B) %**
>
> *Explanation:* The modulus operator (%) computes the remainder when one integer is divided by another.

Q2

What is the output of the following code snippet? `int x = (10, 20, 30);` `printf("%d", x);`

A) 10

B) 20

C) 30

D) Compilation Error

> **Correct Answer: C) 30**
>
> *Explanation:* The comma operator evaluates all operands but returns the rightmost one.

Scanned with CamScanner

Q3

Can math operations be performed on a void pointer?

A) Yes

B) No

> **Correct Answer: B) No**
> *Explanation:* Void pointers do not have a defined size, so pointer arithmetic cannot be directly performed on them.

Q4

Write a ternary expression to find the maximum of two numbers 'a' and 'b'.

> **Answer:** `int max = (a > b) ? a : b;`

Q5

Why does `n++` execute faster than `n+1`?

> **Answer:** The expression `n++` requires a single machine instruction (like INR) to increment the value, whereas `n+1` typically requires multiple instructions (load, add, store).

Scanned with CamScanner

CU Helpdesk — Your Academic Support Community

Click Here to Join WhatsApp Group

08 Lecture 8: Bitwise Operators and Type Conversion

🎯 Learning Objectives

- To learn in detail about bitwise operators, their working at the bit level, and practical applications.
- To understand how type casting (implicit and explicit) works in the C programming language.

8.1 Bitwise Operators ⭐⭐⭐

Mathematical operations within the CPU's arithmetic-logic unit are done at the bit level. Bitwise operators in C allow direct manipulation of data at the binary level. Decimal values are converted into binary sequences, and bitwise operators work on these individual bits.

🔑 Definition

Bitwise operators are operators that perform operations on integer data at the individual bit level. These operations include shifting bits and performing boolean logic on a bit-by-bit basis.

The bitwise operators in C are:

Scanned with CamScanner

- `&` : Bitwise AND
- `|` : Bitwise OR
- `^` : Bitwise XOR
- `~` : Bitwise Complement (NOT)
- `<<` : Left Shift
- `>>` : Right Shift

8.1.1 Bitwise AND (&)

The output of bitwise AND is 1 only if the corresponding bits of both operands are 1. If either bit is 0, the result is 0.

Example: 12 & 25

```
  00001100 (12)
& 00011001 (25)
  --------
  00001000 (8)
```

8.1.2 Bitwise OR (|)

The output of bitwise OR is 1 if at least one corresponding bit of the two operands is 1.

Example: 12 | 25

```
  00001100 (12)
| 00011001 (25)
  --------
  00011101 (29)
```

8.1.3 Bitwise XOR (^)

The result of bitwise XOR is 1 if the corresponding bits of the two operands are opposite (one is 1, the other is 0).

Example: 12 ^ 25

Scanned with CamScanner

```
  00001100 (12)
^ 00011001 (25)
  --------
  00010101 (21)
```

8.1.4 Bitwise Complement (~)

The bitwise complement operator is a unary operator that flips the bits: 1 becomes 0, and 0 becomes 1. For any integer N, the bitwise complement evaluates to `-(N+1)` due to the 2's complement representation of negative numbers.

> ⚠️ **Common Galti**
>
> Log puchte hain `~35` ki value 220 kyu nahi aayi? Kyunki computer negative numbers ko 2's complement format mein store karta hai. Toh MSB 1 hone pe number negative ho jata hai. Hamesha yaad rakhna `~N = -(N+1)`.

8.1.5 Shift Operators (<< and >>)

Shift operators move the bits of a variable to the left or right.

- **Right Shift (>>):** Shifts bits to the right. Equivalent to integer division by powers of 2.
- **Left Shift (<<):** Shifts bits to the left. Equivalent to multiplication by powers of 2.

8.2 Type Casting (Type Conversion) ⭐⭐

Type conversion is the process of converting one data type into another to perform certain operations.

8.2.1 Implicit Type Conversion

Performed automatically by the compiler. When an expression contains mixed data types, the operands of lower rank are automatically promoted to the higher rank datatype to prevent data loss. For example, adding an `int` and a `double` results in a `double`.

Scanned with CamScanner

8.2.2 Explicit Type Conversion

Performed manually by the programmer using the cast operator `(type)`. This allows intentional conversions that the compiler might otherwise flag or execute differently.

Example: `int result = (int)4.5 + (int)4.6;` results in `8`, not `9`.

> 💡 **Senior ka Tip**
>
> Hamesha explicit type casting tab use karo jab tumhe confirm ho ki float se int mein jane se decimal value trim ho jayegi aur tumhe woh chalega.

📌 Quick Revision — Lecture 8

- Bitwise operations work on the binary representations of numbers.
- Bitwise Complement of N is always -(N+1).
- Left shift by 1 multiplies a number by 2; right shift by 1 divides it by 2.
- Implicit type conversion automatically promotes lower data types to higher ones.
- Explicit type conversion requires manual casting by the programmer.

Scanned with CamScanner

Practice Questions — Lecture 8

Q1

What is the output of `printf("%d", ~35);`?

A) 36

B) -36

C) 220

D) -35

> **Correct Answer: B) -36**
>
> *Explanation:* The bitwise complement of N is -(N+1). So, -(35+1) = -36.

Q2

Which bitwise operator is used to set a specific bit to 1?

A) &

B) |

C) ^

D) ~

> **Correct Answer: B) |**
>
> *Explanation:* The OR operator yields 1 if either bit is 1. We can use it with a mask to set specific bits.

Scanned with CamScanner

Q3

How can you detect if two integers have opposite signs without using multiplication?

> **Answer:** You can use the XOR operator. If `(a ^ b) < 0`, the numbers have opposite signs because their most significant bits (sign bits) differ, resulting in a 1 at the MSB position.

Q4

What is the problem with converting a `double` to an `int`?

> **Answer:** When converting a double to an integer, the fractional part is truncated, which can lead to a loss of precision and data.

Q5

What is the output of `printf("a^b = %d", 4^2);`?

A) 12
B) 10
C) 8
D) 6

> **Correct Answer: D) 6**
> *Explanation:* 4 in binary is 100, 2 is 010. XORing them yields 110, which is 6 in decimal.

Scanned with CamScanner

CU Helpdesk — Your Academic Support Community

Click Here to Join WhatsApp Group

09 Lecture 9: Introduction to Expressions

🎯 Learning Objectives

- To understand how expressions evaluate to yield a specific value.
- To learn the concepts of operator precedence and associativity in the C programming language.

9.1 Introduction to Expressions ⭐

An expression is a combination of operators and operands (variables or literal values) that evaluates to a single value of a certain type.

🔑 Definition

An **expression** is a sequence of operands and operators that reduces to a single valid value. It can consist of a single constant, a variable, or complex mathematical formulas.

9.2 Precedence and Associativity ⭐⭐

When there are multiple operators in an expression, they are evaluated according to their precedence and associativity rules.

Scanned with CamScanner

9.2.1 Operator Precedence

Precedence determines which operator is evaluated first. Operators with higher precedence are evaluated before operators with lower precedence.

For example, in the expression `10 + 20 * 30`, multiplication has a higher precedence than addition. Thus, it is evaluated as `10 + (20 * 30)` = 610.

9.2.2 Operator Associativity

When an expression contains operators with the exact same precedence, associativity determines the order of evaluation. Associativity can be either Left-to-Right or Right-to-Left.

For example, `*` and `/` have the same precedence and Left-to-Right associativity. The expression `100 / 10 * 10` is evaluated as `(100 / 10) * 10` = 100.

> ✅ **Ek Baar Padh Lo Bas**
>
> Precedence bataati hai "kaun pehle aayega" (priority). Associativity bataati hai "agar priority same ho gayi toh left se shuru karein ya right se". Bas itna yaad rakhna!

9.2.3 Types of Expressions

- **Integer expressions:** Contain only integer values.
- **Real expressions:** Contain only floating-point values.
- **Mixed-mode arithmetic expressions:** Combine integer and floating-point values.
- **Relational expressions:** Contain relational operators.
- **Logical expressions:** Contain logical operators.

9.3 Dry Runs and Evaluation Steps ⭐⭐⭐

Let us look at a detailed step-by-step evaluation.

Scanned with CamScanner

C PROGRAM

```c
#include <stdio.h>
int main() {
    int a = 2 + 4 + 3 * 5 / 3 - 5;
    printf("Value of a = %d\n", a);
    return 0;
}
```

Step	Operation Executed	Expression Status
1	Evaluate Multiplication (highest precedence left-to-right)	2 + 4 + 15 / 3 - 5
2	Evaluate Division	2 + 4 + 5 - 5
3	Evaluate First Addition	6 + 5 - 5
4	Evaluate Second Addition	11 - 5
5	Evaluate Subtraction	6

Output:

```
Value of a = 6
```

Scanned with CamScanner

📌 Quick Revision — Lecture 9

- Precedence resolves evaluation order when operators differ in priority.
- Associativity resolves evaluation order when operators share the same priority.
- Round brackets `()` have the highest precedence.
- The comma operator `,` has the lowest priority.
- Postfix increment evaluates left-to-right, whereas prefix increment evaluates right-to-left.

📝 Practice Questions — Lecture 9

Q1

Which brackets have the highest precedence in C?

A) {} (Curly)
B) [] (Square)
C) () (Round)
D) <> (Angle)

> **Correct Answer: C) () (Round)**
> *Explanation:* Round brackets are always evaluated first to override standard precedence rules.

Scanned with CamScanner

Q2

What is the output of `double b = 5 % 3 & 4 + 5 * 6; printf("%lf", b);`?

> **Answer: 2.000000**
> *Explanation:* The precedence rules order the operators as follows: `*`, `%`, `+`, and finally bitwise `&`.

Q3

Does the comma operator have the highest or lowest precedence?

> **Answer: Lowest**
> *Explanation:* The comma operator executes after all other operators in the expression have evaluated.

Q4

What is the difference between precedence and associativity?

> **Answer:** Operator precedence determines which operator evaluates first among differing operators. Associativity is used as a tie-breaker when two operators share the same precedence level, defining whether evaluation flows left-to-right or right-to-left.

Scanned with CamScanner

Q5

What is the remainder for `5.0 % 2`?

> **Answer: Compilation Error**
>
> *Explanation:* The modulus operator (%) cannot be applied to floating-point numbers in C. It strictly requires integer operands.

Scanned with CamScanner

CU Helpdesk — Your Academic Support Community

Click Here to Join WhatsApp Group

10 Lecture 10: Decision Control Statements

🎯 Learning Objectives

- To learn where and how to use decision statements in C programming.
- To understand the various types of decision control structures.
- To implement conditional logic to control the flow of program execution.

10.1 What is a Decision Making Statement? ⭐

In the C programming language, the program execution flow is typically sequential, meaning it runs line by line from top to bottom starting from the `main()` function. However, this straight-line execution is not always suitable for solving complex problems. Often, we need to make decisions and execute specific blocks of code only if certain conditions are met, while skipping others.

For example, if we want to determine whether a student has passed or failed, we must check if their marks are greater than the passing threshold. If the condition is true, we display "Pass"; otherwise, we display "Fail". To implement this kind of logic, C provides decision-making statements.

Scanned with CamScanner

🔑 Definition

Decision control statements (or conditional statements) are programming constructs that allow you to control the flow of execution based on specific conditions. They evaluate a test expression to true or false, determining which segment of code will be executed and which will be ignored.

The primary decision-making statements in C are:

- `if` statement
- `switch` statement

10.2 The Simple `if` Statement ⭐

The `if` statement is the simplest form of decision control. It evaluates a single test condition. If the condition evaluates to true (non-zero), the block of statements enclosed within the `if` block is executed. If the condition evaluates to false (zero), the block is skipped entirely.

SYNTAX

```
        if (condition)
{
    // Statements to be executed if condition is true
}
```

💡 Senior ka Tip

Bro isko aise samjho: Agar tumhare paas VIP pass (condition true) hai, toh hi tum club (if block) ke andar jaa sakte ho, warna bahar se hi aage nikal jao (skip block).

Scanned with CamScanner

Example: Displaying a message if a number is greater

C PROGRAM

```c
#include <stdio.h>

void main()
{
    int x = 15;
    int y = 10;

    // Checking if x is strictly greater than y
    if (x > y)
    {
        printf("x is greater than y\n");
    }
}
```

📺 **Output:**

```
x is greater than y
```

Dry Run:

Variable `x`	Variable `y`	Condition `(x > y)`	Action
15	10	15 > 10 (True)	Enter the `if` block, execute `printf`.

Scanned with CamScanner

> ⚠️ **Common Galti**
>
> If ke aage semicolon `if (condition);` galti se bhi mat lagana! Agar laga diya toh condition check hone ke baad C usko wahi khatam kar dega aur block ke andar ka code hamesha chalega.

10.3 The `if-else` Statement ⭐⭐

The `if-else` statement extends the simple `if` statement. It provides a two-way branch. If the test condition is true, the statements within the `if` block are executed. If the test condition is false, the statements within the `else` block are executed.

SYNTAX

```
        if (condition)
{
    // Block 1: Executed if true
}
else
{
    // Block 2: Executed if false
}
```

Scanned with CamScanner

Example: Checking if a number is positive or negative

C PROGRAM

```c
#include <stdio.h>

void main()
{
    int a = -7;

    // Check if the number is greater than zero
    if (a > 0)
    {
        printf("Number is positive\n");
    }
    else
    {
        printf("Number is negative\n");
    }
}
```

Output:

```
Number is negative
```

Scanned with CamScanner

Example: Checking whether an integer is Odd or Even

C PROGRAM

```c
#include <stdio.h>

int main()
{
    int number = 3;

    // Modulo operator returns remainder
    if (number % 2 == 0)
    {
        printf("%d is an even integer.\n", number);
    }
    else
    {
        printf("%d is an odd integer.\n", number);
    }
    return 0;
}
```

📺 **Output:**

```
3 is an odd integer.
```

10.4 Nested `if-else` Statement ⭐⭐⭐

When an `if` or `if-else` block contains another `if` or `if-else` block inside it, it is known as a nested `if` statement. This is highly useful when a decision depends on multiple sequential conditions.

Scanned with CamScanner

Example: Finding the maximum among three numbers

C PROGRAM

```c
#include <stdio.h>

void main()
{
    int a = 5, b = 10, c = 15;

    // Outer if
    if (a > b)
    {
        // Inner if 1
        if (a > c)
        {
            printf("a is the greatest\n");
        }
        else
        {
            printf("c is the greatest\n");
        }
    }
    else
    {
        // Inner if 2
        if (b > c)
        {
            printf("b is the greatest\n");
        }
        else
        {
            printf("c is the greatest\n");
        }
    }
}
```

Scanned with CamScanner

📺 **Output:**

```
c is the greatest
```

Dry Run:

Condition Evaluated	Result	Next Step
`a > b` (5 > 10)	False	Jump to the outer `else` block.
`b > c` (10 > 15)	False	Jump to the inner `else` block.
Execute	N/A	Prints "c is the greatest".

10.5 `if-else if` Ladder ⭐⭐

The `else-if` ladder is utilized when a multipath decision is required. The conditions are evaluated sequentially from top to bottom. The moment a condition evaluates to true, the block associated with it is executed, and the remainder of the ladder is bypassed.

✅ Ek Baar Padh Lo Bas

Ladder matlab seedhi. C compiler upar se ek ek condition check karte hue neeche aayega. Jaise hi koi condition true mili, uske andar ka kaam karega aur poori ladder ke bahar aa jayega!

Scanned with CamScanner

Example: Checking divisibility logic

C PROGRAM

```c
#include <stdio.h>

void main()
{
    int a = 16;

    if (a % 5 == 0 && a % 8 == 0)
    {
        printf("Divisible by both 5 and 8\n");
    }
    else if (a % 8 == 0)
    {
        printf("Divisible by 8\n");
    }
    else if (a % 5 == 0)
    {
        printf("Divisible by 5\n");
    }
    else
    {
        printf("Divisible by none\n");
    }
}
```

📺 **Output:**

```
Divisible by 8
```

Scanned with CamScanner

📌 Quick Revision — Lecture 10

- **Decision statements** control program execution conditionally.
- **if statement:** Executes block only if condition is true.
- **if-else statement:** Two paths; one for true, one for false.
- **Nested if-else:** Multi-level testing by placing conditionals inside one another.
- **if-else if ladder:** Sequential testing to select one valid block among many.

📝 Practice Questions — Lecture 10

MCQ Q1

Choose a correct C Statement using the IF Conditional Statement.

A) if(condition) { }
B) if(condition) { } else { }
C) if(condition1) { } else if(condition2) { } else { }
D) All of the above

> **Correct Answer: D) All of the above**
> *Explanation:* The syntax for simple if, if-else, and else-if ladders are all valid conditional constructs in C.

Scanned with CamScanner

MCQ Q2

What is the output of the following code snippet?

```
int main() {
    if( 4 > 5 ) {
        printf("Hurray..\n");
    }
    printf("Yes");
    return 0;
}
```

A) Yes

B) Hurray.. Yes

C) Hurray..

D) Compiler error

> **Correct Answer: A) Yes**
>
> *Explanation:* The condition `4 > 5` is false, so the `if` block is skipped. The program proceeds to print "Yes".

Scanned with CamScanner

MCQ Q3

What is the output of the following C code?

```
int main() {
    if("abc") {
        printf("India\n");
    }
    if('c') {
        printf("Honey\n");
    }
    printf("ZING");
    return 0;
}
```

A) ZING

B) Honey ZING

C) India ZING

D) India Honey ZING

> **Correct Answer: D) India Honey ZING**
>
> *Explanation:* In C, any non-zero value or valid string/character literal is considered true. Both "abc" and 'c' evaluate to true. Therefore, all three print statements are executed.

Scanned with CamScanner

MCQ Q4

What will be the output of the following C code?

```
int main () {
    int x = 5;
    if (x < 1);
        printf("Hello");
}
```

A) Nothing

B) Run time error

C) Hello

D) Varies

> **Correct Answer: C) Hello**
>
> *Explanation:* The semicolon immediately following the `if (x < 1)` statement terminates the `if` construct immediately. Thus, the `printf("Hello");` statement is independent and executes regardless of the condition.

Scanned with CamScanner

Write a Program Q5

Design a program to read the age of a candidate and determine whether they are eligible to cast their vote.

Age threshold: 18 years

Correct Answer:

```
#include <stdio.h>
int main() {
    int age;
    printf("Enter the age of the person: ");
    scanf("%d", &age);
    if (age >= 18) {
        printf("Eligible for voting\n");
    } else {
        printf("Not eligible for voting\n");
    }
    return 0;
}
```

Scanned with CamScanner

CU Helpdesk — Your Academic Support Community

Click Here to Join WhatsApp Group

11 Lecture 11: Switch Statements

🎯 Learning Objectives

- To learn the structural syntax and behavior of the `switch` case statement.
- To understand how the `break` and `default` keywords function within a switch.
- To build menu-driven programs using the switch control structure.

11.1 Syntax of Switch Statement ⭐

The `switch` statement allows a variable to be tested for equality against a list of values. Each value is called a case, and the variable being switched on is checked for each case sequentially. When a match is found, the corresponding block of code executes.

Scanned with CamScanner

SYNTAX

```
        switch (expression)
{
    case value1:
        // statements
        break;

    case value2:
        // statements
        break;

    default:
        // default statements
}
```

> 🔑 **Definition**
>
> The `switch` statement evaluates a given test expression and branches program execution to the matching `case` block. If no case values match the test expression, it falls back to executing the optional `default` block.

11.2 Role of the `break` Statement ⭐⭐

The `break` statement is crucial for the proper execution of a switch block. When the compiler encounters a `break`, it immediately halts the switch execution and takes the control flow outside the switch body.

> ⚠️ **Common Galti**
>
> Bhai agar `break` lagana bhool gaye, toh C compiler matching case ke baad wale SAARE cases bhi execute kar dega (iske bolte hain Fall-Through error).

Scanned with CamScanner

Example: Weekday without Break (Demonstrating Fall-Through)

C PROGRAM

```c
#include <stdio.h>

int main()
{
    int v = 2;
    switch(v)
    {
        case 1: printf("Case 1\n");
        case 2: printf("Case 2\n");
        case 3: printf("Case 3\n");
        default: printf("Default Case\n");
    }
    return 0;
}
```

Output:

```
Case 2 Case 3 Default Case
```

Notice how after Case 2 matched, the lack of a break statement caused Case 3 and the Default Case to execute as well.

Scanned with CamScanner

Example: Using Break correctly

C PROGRAM

```c
#include <stdio.h>

int main()
{
    int v = 2;
    switch(v)
    {
        case 1:
            printf("Case 1\n");
            break;
        case 2:
            printf("Case 2\n");
            break;
        case 3:
            printf("Case 3\n");
            break;
        default:
            printf("Default Case\n");
    }
    return 0;
}
```

📺 **Output:**

```
Case 2
```

11.3 Important Rules for Switch Expressions ⭐⭐⭐

When utilizing the switch statement, there are several strict constraints on how the syntax can be formulated:

Scanned with CamScanner

1. **Order of cases:** The case values do not have to be written in ascending or any specific order.
2. **Character cases:** You can use characters for cases (e.g., `case 'A':`), which evaluates to their respective ASCII integer values.
3. **Constant evaluation:** The test expression inside the `switch` parentheses must return a constant integer value. Variables are not allowed in the case labels. Valid examples include `switch(1+2)`, but invalid examples would include `switch(a+b)` in the case label.
4. **Duplicate values:** Duplicate case values are strictly prohibited. Writing `case 1:` twice will yield a compilation error.
5. **Optional default:** The `default` block is optional. The program functions perfectly without it, but it handles unexpected inputs gracefully.

🔥 Yeh Exam Mein Zaroor Aayega

Teachers love asking if `float` or `string` can be used in a switch. Yaad rakhna, switch only works with integers and characters. Floats (like 3.14) or Strings (like "hello") will give compile-time errors.

Scanned with CamScanner

Example: Menu Driven Calculator

C PROGRAM

```c
#include <stdio.h>

void main()
{
    int num1 = 10, num2 = 2, opt = 3;
    // Assume user inputted 10, 2 and chose option 3 for Multiplication

    switch(opt)
    {
        case 1:
            printf("Addition: %d\n", num1 + num2);
            break;
        case 2:
            printf("Subtraction: %d\n", num1 - num2);
            break;
        case 3:
            printf("Multiplication: %d\n", num1 * num2);
            break;
        case 4:
            if (num2 == 0) {
                printf("Divide by zero error\n");
            } else {
                printf("Division: %d\n", num1 / num2);
            }
            break;
        default:
            printf("Input correct option\n");
            break;
    }
}
```

Scanned with CamScanner

📺 **Output:**

```
Multiplication: 20
```

📌 Quick Revision — Lecture 11

- **switch** matches an expression against multiple discrete `case` values.
- **break** is used to exit the switch block preventing execution of subsequent cases.
- **default** handles fallback scenarios when no case matches.
- Only **integers and characters** are valid test expressions. Duplicate cases are disallowed.

📝 Practice Questions — Lecture 11

MCQ Q1

Which of the following data types cannot be checked in a switch-case statement in C?

A) Character
B) Integer
C) Float
D) enum

> **Correct Answer: C) Float**
> *Explanation:* The switch statement requires expression evaluation that resolves to a discrete integral value. Floating point numbers are not permitted.

Scanned with CamScanner

MCQ Q2

Can we use string variables in a switch test condition? (e.g. `switch("hello")`)

A) Yes

B) No

> **Correct Answer: B) No**
>
> *Explanation:* Switch statements only support single character and integer matching.

Code Output Q3

What will be the output of the following program?

```
#include <stdio.h>
void main() {
    int a=10;
    switch(a) {
        case 5+5:
            printf("Hello\n");
        default:
            printf("OK\n");
    }
}
```

A) Hello

B) OK

C) Hello \n OK

D) Error

> **Correct Answer: C) Hello \n OK**
>
> *Explanation:* The expression `case 5+5:` evaluates to `case 10:`, which matches variable `a`. "Hello" is printed. Since there is no `break`, control falls through to print "OK" as well.

Scanned with CamScanner

Write a Program Q4

Write a C program to read a weekday number (0-6) and print the weekday name using switch.

0 = Sunday, 1 = Monday, etc.

Correct Answer:

```
#include <stdio.h>
int main() {
    int wDay = 3;
    switch(wDay) {
        case 0: printf("Sunday\n"); break;
        case 1: printf("Monday\n"); break;
        case 2: printf("Tuesday\n"); break;
        case 3: printf("Wednesday\n"); break;
        case 4: printf("Thursday\n"); break;
        case 5: printf("Friday\n"); break;
        case 6: printf("Saturday\n"); break;
        default: printf("Invalid weekday\n");
    }
    return 0;
}
```

Output: `Wednesday`

Scanned with CamScanner

Write a Program Q5

Write a C program to check whether a given character is a VOWEL or CONSONANT using switch.

Check both upper and lowercase vowels.

Correct Answer:

```
#include <stdio.h>
int main() {
    char ch = 'E';
    switch(ch) {
        case 'A': case 'E': case 'I': case 'O': case 'U':
        case 'a': case 'e': case 'i': case 'o': case 'u':
            printf("%c is a VOWEL.\n", ch);
            break;
        default:
            printf("%c is a CONSONANT.\n", ch);
    }
    return 0;
}
```

Scanned with CamScanner

CU Helpdesk — Your Academic Support Community

Click Here to Join WhatsApp Group

12 Lecture 12: Conditional Operator and Goto

🎯 Learning Objectives

- To understand how the `goto` statement manipulates control flow via forward and backward jumping.
- To master the syntax and application of the ternary conditional operator `(?:)`.
- To convert extensive `if-else` logic into concise single-line statements.

12.1 Introduction to the `goto` Statement ⭐⭐

The `goto` statement allows a programmer to unconditionally transfer the flow of execution to a specific label located anywhere within the same function. It is often used to alter the flow entirely, jumping over portions of code or repeating blocks to create a looping-like structure.

🔑 Definition

A `goto` statement performs an unconditional jump from the point where the `goto` is called to a marked identifier called a **label**.

Scanned with CamScanner

SYNTAX

```
        goto label_name;
...
label_name:
    // Code statements to execute after jump
```

Forward Jump: When the label is defined *after* the `goto` call.

Backward Jump: When the label is defined *before* the `goto` call (useful for simulating loops).

> ⚠️ **Common Galti**
>
> Modern programming practices mein `goto` ka zyada use mana jata hai "Spaghetti code" (uljha hua code) banane ke liye. Tabhi use karo jab bohot zaroori ho, like breaking out of deeply nested loops.

Scanned with CamScanner

Example: Print table of 10 using backward jump

C PROGRAM

```c
#include <stdio.h>

void main()
{
    int i = 1;
    int num = 10;

table_label:  // Label declaration
    printf("%d x %d = %d\n", num, i, num * i);
    i++;

    if (i <= 5)
    {
        goto table_label; // Unconditional jump backward
    }
}
```

📺 **Output:**

```
10 x 1 = 10 10 x 2 = 20 10 x 3 = 30 10 x 4 = 40 10 x 5 = 50
```

12.2 Conditional (Ternary) Operator `(?:)` ⭐⭐

The conditional operator is closely related to the `if-else` construct. It implements the exact same logic but uses fewer lines of code. It is highly efficient when assigning values based on a single, simple condition.

Scanned with CamScanner

SYNTAX

```
(Condition) ? Expression1 : Expression2;
```

Because it operates on three distinct operands, it is commonly referred to as the **Ternary Operator**.

Explanation: If the `Condition` is true, it evaluates and executes `Expression1`. Otherwise, it executes `Expression2`.

> 💡 **Senior ka Tip**
>
> Agar `if-else` mein bas ek hi line ka kaam hai (jaise variable assign karna), toh Ternary operator use karo. Code ek dum pro lagega! `ans = (x > y) ? x : y;`

Example: Find max of two numbers

C PROGRAM

```c
#include <stdio.h>

int main()
{
    int n1 = 5, n2 = 10, max;

    // Evaluates if n1 is greater than n2
    max = (n1 > n2) ? n1 : n2;

    printf("Maximum is %d\n", max);
    return 0;
}
```

Scanned with CamScanner

Output:

```
Maximum is 10
```

Example: Nesting Ternary to find Max of three numbers

C PROGRAM

```c
#include <stdio.h>

int main()
{
    int a = 5, b = 20, c = 15, big;

    // Nested ternary operators
    big = (a > b) ? ( (a > c) ? a : c ) : ( (b > c) ? b : c );

    printf("Maximum among three is %d\n", big);
    return 0;
}
```

Output:

```
Maximum among three is 20
```

Scanned with CamScanner

📌 Quick Revision — Lecture 12

- **goto statement:** Shifts execution control unconditionally to a defined label.
- A backward `goto` can construct looping behavior, while forward `goto` skips logic blocks.
- **Ternary Operator (?:)** acts as a compact alternative to basic `if-else` structures.
- Ternary operators improve concise readability but can become complex when deeply nested.

📝 Practice Questions — Lecture 12

MCQ Q1

What is the alternative name for the `?:` operator in C?

A) Question Mark Colon Operator
B) Ternary Operator
C) Conditional Logic Operator
D) Binary Operator

> **Correct Answer: B) Ternary Operator**
> *Explanation:* It operates on three operands, hence the name ternary operator.

Scanned with CamScanner

MCQ Q2

Can a `goto` statement be used to jump from the `main` function into another function entirely?

A) True

B) False

> **Correct Answer: B) False**
>
> *Explanation:* The `goto` label scope is strictly limited to the function it is defined within. You cannot jump between different functions.

Code Output Q3

What is the output of the C Program?

```
int main() {
    int a=0;
    a = (5 > 2) ? 8 : 10;
    printf("%d", a);
    return 0;
}
```

A) 5

B) 2

C) 8

D) 10

> **Correct Answer: C) 8**
>
> *Explanation:* The condition `5 > 2` is true. Therefore, the ternary operator resolves to `8`, assigning it to `a`.

Scanned with CamScanner

Write a Program Q4

Write a C program to check if an entered integer is Even or Odd using the Conditional (Ternary) operator.

Test with input: 11

Correct Answer:

```
#include <stdio.h>
int main() {
    int num = 11;
    (num % 2 == 0) ? printf("The number is EVEN\n") : printf("The number is ODD\n");
    return 0;
}
```

Output: `The number is ODD`

Scanned with CamScanner

Write a Program Q5

Design a program using the `goto` statement to print the factorial of a number.

Test with input: 5

Correct Answer:

```
#include <stdio.h>
int main() {
    int fact = 1, i = 1, num = 5;

loop_start:
    fact = fact * i;
    i++;
    if (i <= num) {
        goto loop_start;
    }

    printf("Factorial is %d\n", fact);
    return 0;
}
```

Output: `Factorial is 120`

Scanned with CamScanner